

CSA1208 — COMPUTER ARCHITECTURE
COURSE ASSIGNMENT REPORT

Design and Performance Evaluation of an
Intelligent Traffic Management System (ITMS)

Submitted by

Name: M.Venkat lokesh

Register No.: 192412377

Guided by

Dr. Rashmita Khilar

Course Outcomes Covered: CO1, CO4, CO5

SDG Mapping: SDG 9, SDG 11, SDG 13

Department of Computer Science and Engineering

Table of Contents

Abstract

1. Introduction

2. Problem Definition

3. Real-World Application

4. Objectives

5. Requirements

6. Proposed ITMS Architecture and Data Flow

7. Student 1 — Processor Performance

8. Student 2 — Memory and Cache Optimization

9. Student 3 — Traffic Workload and System-Level Evaluation

10. I/O Architecture

11. Implementation

12. Simulation Methodology

13. Overall Performance Comparison and Engineering Recommendation

14. Power, Cost, Reliability and Scalability

15. Sustainable Development Goal (SDG) Analysis

16. Limitations

17. Future Scope

18. Conclusion

19. References

20. Student Contribution

21. Appendix A — Core Source Code (src/main.py)

Abstract

This report presents the design, simulation-based implementation and performance evaluation of a computer architecture for an Intelligent Traffic Management System (ITMS). The ITMS is designed to process real-time data from traffic cameras, vehicle sensors and road-monitoring devices at a signalised intersection, to detect congestion, optimize traffic-signal timing, reduce average waiting time and give priority movement to emergency vehicles, while remaining energy-efficient and scalable to a city-wide deployment.

The project is organized into three connected parts that together form one final architecture: Student 1 analyzed processor performance across three candidate architecture families (ARM, x86-64 and RISC-V) using instruction count, CPI, clock cycles, execution time and utilization; Student 2 analyzed the memory hierarchy and cache organization (L1/L2/L3) using hit rate, miss rate, Average Memory Access Time (AMAT), bandwidth and throughput; and Student 3 modelled three traffic workloads — Normal, Peak-Hour Congestion and Emergency Vehicle — and evaluated the combined processor-and-cache architecture under each.

A single Python simulation program (`src/main.py`) implements all three parts, computes every quoted number using the standard performance equations, and generates the comparison tables and graphs used throughout this report. Based on the simulation results, the RISC-V (SiFive U74-class) processor paired with the Optimized L1/L2/L3 cache configuration is recommended: it gives the best power-and-cost-adjusted performance while keeping worst-case processing latency under 1 millisecond per frame across all three workloads, well inside the assumed 100 ms real-time budget. All results in this report are simulation-based; no physical hardware was measured. Section 15 states the assumed simulation parameters explicitly.

1. Introduction

Urban road intersections generate continuous streams of visual and sensor data — vehicle counts, queue lengths, pedestrian activity and emergency-vehicle signals — that must be processed within tight real-time deadlines to keep traffic flowing safely. Static, fixed-timing traffic signals cannot react to this data, which leads to unnecessary congestion, wasted fuel and delayed emergency response. An Intelligent Traffic Management System (ITMS) addresses this by continuously ingesting camera, sensor and road-monitoring data and using it to drive adaptive signal control.

From a Computer Architecture perspective, the interesting engineering problem is not the traffic-control policy itself but the underlying hardware: which processor instruction-set architecture, memory hierarchy and I/O strategy can sustain this real-time image- and sensor-processing workload reliably, at low power and low cost, and scale from one intersection to an entire city. This report analyzes that problem using the standard computer-architecture performance equations (instruction count, CPI, clock cycles, execution time, AMAT, bandwidth and throughput), applied to a Python simulation of the ITMS workload, and produces an evidence-based architecture recommendation.

2. Problem Definition

Given real-time data streams from traffic cameras, vehicle sensors and road-monitoring devices at a signalised intersection, the problem is to select and justify a computer architecture (processor family, memory/cache hierarchy and I/O mechanism) that can: (a) process each camera frame and sensor batch within a real-time deadline, (b) sustain this processing across Normal, Peak-Hour and Emergency traffic conditions without missing the deadline, (c) minimize energy consumption and hardware cost, and (d) scale to additional intersections in a smart-city deployment — while explicitly separating simulation-based estimates from real hardware measurements, since no physical hardware was benchmarked for this assignment.

3. Real-World Application

The chosen application is a roadside Intelligent Traffic Management unit for a single signalised urban intersection, generalizable to a network of intersections in a smart-city traffic-control system. It is scoped as a simulation study of the compute architecture that would sit inside such a roadside unit (or an edge-compute box serving a small cluster of intersections), rather than a claim about any specific commercial traffic-control product.

4. Objectives

- Analyze and compare candidate processor architectures (ARM, x86-64, RISC-V) for real-time ITMS workloads using instruction count, CPI, clock cycles, execution time and utilization.
- Design and evaluate a memory/cache hierarchy (L1/L2/L3) suited to frequently-accessed traffic-camera and sensor data, using hit rate, miss rate, AMAT, bandwidth and throughput.
- Model three realistic traffic workloads (Normal, Peak-Hour, Emergency) and evaluate the combined processor-and-cache architecture under each.
- Implement a working Python simulation that generates every number, table and graph quoted in this report.

- Recommend an optimized architecture, justified by power, cost, reliability and scalability analysis, and connect the recommendation to SDG 9, SDG 11 and SDG 13.

5. Requirements

5.1 Functional Requirements

- FR1: Acquire real-time data from traffic cameras, vehicle sensors and road-monitoring devices.
- FR2: Process each camera frame and sensor batch to detect congestion and vehicle counts.
- FR3: Detect emergency-vehicle presence and trigger a priority signal-override path.
- FR4: Compute and log processor and memory-hierarchy performance metrics for each workload.
- FR5: Generate comparison tables and graphs for processor, cache and workload evaluation.

5.2 Non-Functional Requirements

- NFR1 (Real-time): each reference frame/sensor batch must complete within a 100 ms processing budget.
- NFR2 (Energy efficiency): average power draw should remain low enough for continuous roadside operation on a modest power budget.
- NFR3 (Scalability): the architecture must extend from one intersection to a multi-intersection, city-wide deployment without a redesign.
- NFR4 (Reliability): the system must degrade gracefully under sensor, processor or communication failure.

5.3 Constraints

- All results are simulation-based (Simulation-Based Evaluation); no physical hardware was measured.
- Clock frequencies, CPI, cache hit rates and traffic-load figures are assumed, literature-informed simulation parameters, not vendor benchmark results.
- Student names and register numbers are left as placeholders per assignment instructions.

6. Proposed ITMS Architecture and Data Flow

The proposed architecture is a layered pipeline: sensing devices (cameras, vehicle sensors, road-monitoring devices) feed a data-acquisition stage over memory-mapped I/O and DMA; a single embedded-class processor executes preprocessing, traffic analysis and decision logic; a three-level cache hierarchy (L1/L2/L3) keeps frequently accessed frame and traffic-control data close to the processor; and a decision engine drives the traffic-signal controller, with a dedicated emergency-vehicle path that can override normal signal timing. Figure 1 shows the overall block architecture and Figure 2 shows the corresponding data-processing flow.

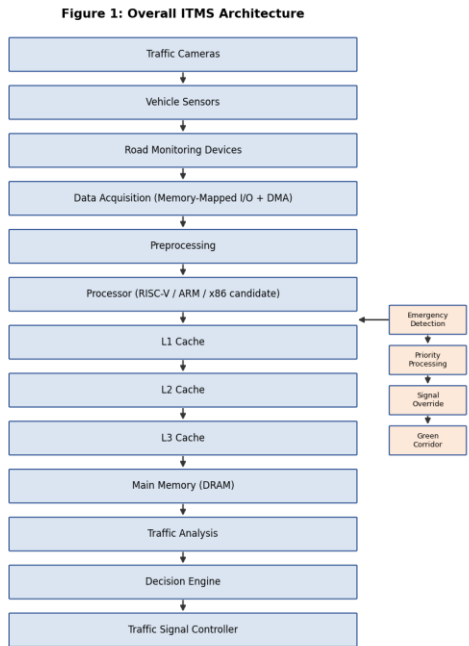


Figure 1: Overall ITMS Architecture

Figure 2: ITMS Data Processing Flow

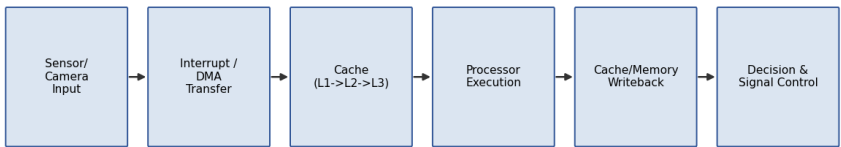


Figure 2: ITMS Data Processing Flow

7. Student 1 — Processor Performance

Three candidate processor architectures were modelled against a common reference workload of 45,000,000 instructions per reference unit (one fused camera frame plus its associated sensor batch, processed through the perception-to-decision pipeline), within an assumed 100 ms real-time budget. This instruction count is held constant across all three processors so the comparison isolates architectural differences (CPI, clock frequency) rather than workload differences.

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{CPI}$$

$$\text{Execution Time} = \text{Clock Cycles} / \text{Clock Frequency}$$

$$\text{Processor Utilization} = (\text{Execution Time} / \text{Available Time}) \times 100$$

Architecture	ISA	Clock (GHz)	CPI	Clock Cycles	Exec. Time (ms)	MIPS	Utilization (%)	Power (W)	Cost (USD)
ARM Cortex-A72	ARMv8-A (64-bit RISC)	1.8	1.20	54,000,000	30.000	1500.0	30.00	4.0	35
Intel Atom x6000E	x86-64 (CISC)	1.6	1.60	72,000,000	45.000	1000.0	45.00	6.5	55
SiFive U74	RISC-V (RV64GC)	1.4	1.15	51,750,000	36.964	1217.4	36.96	2.5	20

Table 1: Processor Architecture Comparison (Simulation-Based)

All three candidates meet the 100 ms real-time budget with significant headroom. ARM achieves the lowest execution time (30.0 ms) due to its higher clock frequency and low CPI. RISC-V trades a lower clock frequency for the lowest CPI and by far the lowest power draw (2.5 W) and lowest relative cost (USD 20), making it the strongest candidate once power and cost are weighted alongside raw speed for a roadside, mains- or solar-powered deployment. x86-64 has the weakest utilization headroom and the highest power draw of the three, driven by its higher CPI (1.60) from its variable-length CISC instruction decoding.

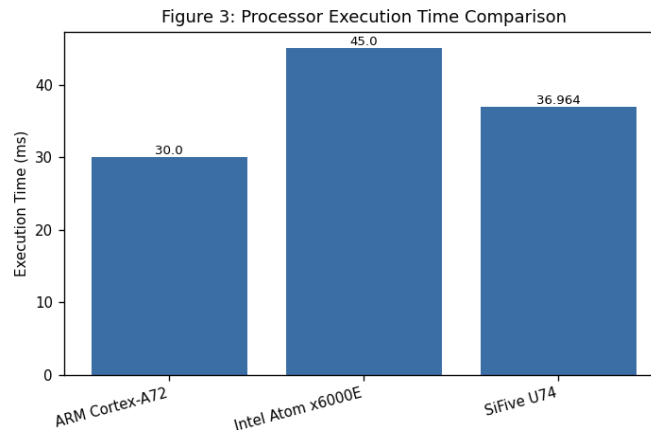


Figure 3: Processor Execution Time Comparison

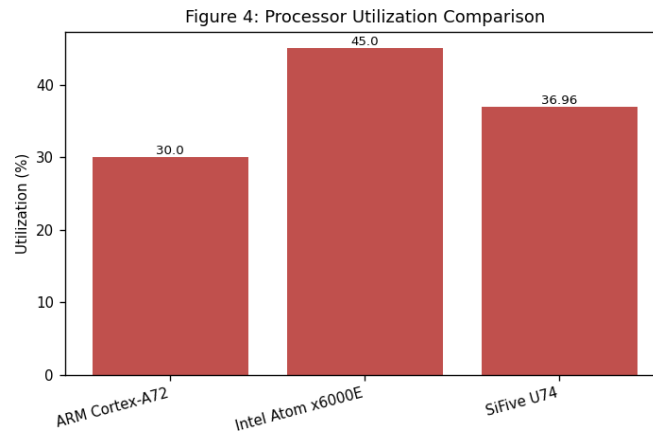


Figure 4: Processor Utilization Comparison

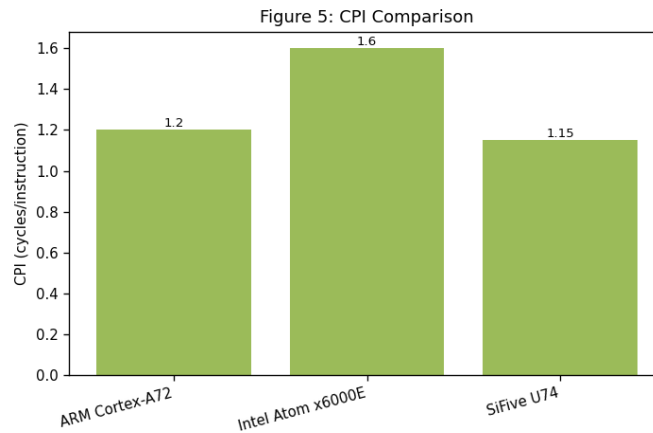


Figure 5: CPI Comparison

Recommendation (Student 1): the RISC-V (SiFive U74-class, RV64GC) architecture is recommended for the ITMS processor, based on its lowest CPI, lowest power consumption and lowest relative cost while still comfortably meeting the real-time budget.

8. Student 2 — Memory and Cache Optimization

Two cache/memory-hierarchy configurations were compared: a Baseline configuration with a small L1 and no L3, and an Optimized configuration with a larger, ITMS-tuned three-level L1/L2/L3 hierarchy sized for the working set of recent camera frames and hot traffic-control tables.

$$\text{Miss Rate} = 1 - \text{Hit Rate}$$

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Configuration	L1 (KB)	L2 (KB)	L3 (KB)	Hit Rate	Miss Rate	AMAT (ns)	Bandwidth (GB/s)	Throughput (Mops/s)
Baseline (small L1, no L3)	32	256	0	0.85	0.15	14.5	8.0	69.0
Optimized (L1+L2+L3, ITMS-tuned)	64	1024	8192	0.94	0.06	6.4	25.6	156.2

Table 2: Cache Configuration Comparison (Simulation-Based)

Adding an L3 cache and increasing L1/L2 capacity raises the overall hit rate from 0.85 to 0.94 and more than halves AMAT (14.5 ns to 6.4 ns), because a much larger fraction of frame-buffer and traffic-control-table accesses are served from on-chip cache instead of paying the 90 ns main-memory miss penalty. The Optimized configuration also more than triples effective bandwidth and throughput, which directly benefits camera-frame processing where each frame touches a large, reused pixel/feature working set.

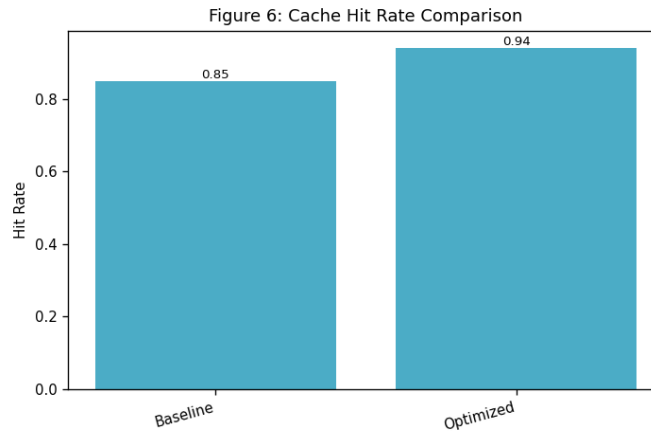


Figure 6: Cache Hit Rate Comparison

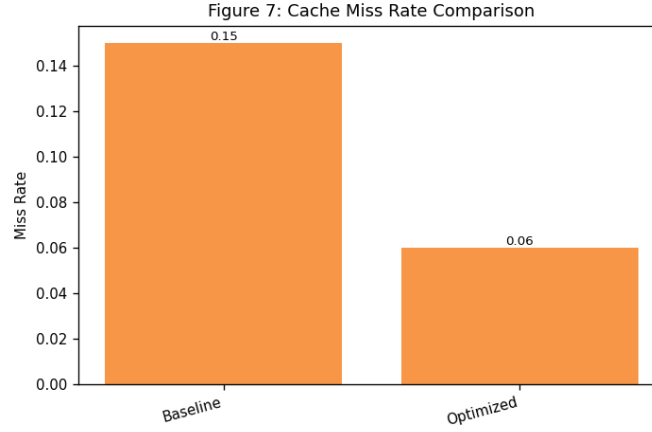


Figure 7: Cache Miss Rate Comparison

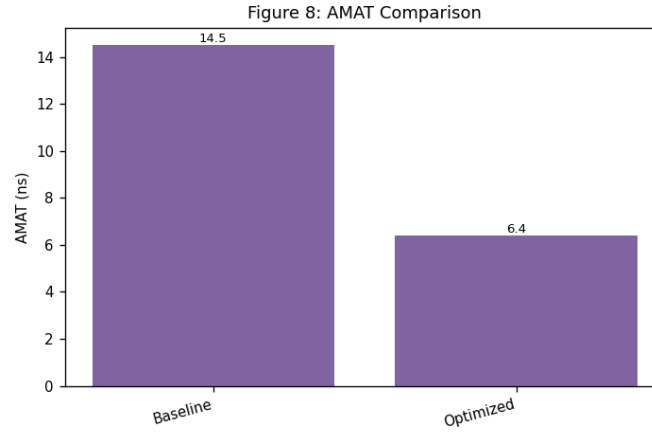


Figure 8: AMAT Comparison

Recommendation (Student 2): the Optimized L1/L2/L3 configuration is recommended, as it reduces AMAT by 56% and increases hit rate by 9 percentage points over the Baseline, directly benefiting the frame- and sensor-heavy access pattern of the ITMS workload.

8.1 Memory Technology Notes

The cache levels (L1/L2/L3) are assumed to be built from SRAM for its low latency, at the cost of lower density than DRAM. Main memory uses DRAM for its higher density and lower cost per bit despite the added refresh overhead reflected in the miss-penalty figure. Any persistent local storage (e.g. logging or offline model updates) would use NAND Flash for non-volatility and cost-effective bulk storage; MRAM and RRAM are noted as emerging non-volatile alternatives that could eventually replace part of the DRAM/Flash split for faster, lower-power persistent state, but were not modelled quantitatively in this simulation. Virtual memory with paging is assumed at the OS level to isolate the perception, decision and logging processes, but is not a performance bottleneck at this workload scale and is therefore not separately simulated.

9. Student 3 — Traffic Workload and System-Level Evaluation

Three workload scenarios were modelled on the recommended architecture (RISC-V processor + Optimized cache): Normal Traffic, Peak-Hour Congestion and Emergency Vehicle. Each scenario combines a vehicle count, sensor-event rate and camera frame rate into an instruction workload, which is then evaluated for processor utilization, cache behaviour, per-frame latency, throughput, average power and energy.

Metric	Normal	Peak-Hour	Emergency
Vehicles / min	40	160	100
Sensor events / min	120	480	300
Camera FPS	15	25	30
Instruction workload (instr/s)	13,508,333	22,533,333	33,020,833
CPI	1.20	1.35	1.25
Processor utilization (%)	1.16	2.17	2.95
Cache hit rate	0.94	0.90	0.92
AMAT (ns)	6.4	10.0	8.2
Processing latency (ms/frame)	0.778	0.878	0.990
Throughput (frames/s)	1285.63	1139.14	1009.75
Energy (J/min)	90.7	91.3	91.8

Table 3: Traffic Workload Comparison (Simulation-Based)

Peak-Hour Congestion increases instruction workload by roughly 67% over Normal traffic (more vehicles and sensor events), which lowers the cache hit rate slightly (0.94 to 0.90) due to greater working-set pressure and raises AMAT accordingly. The Emergency scenario carries the heaviest per-frame compute cost, because emergency-detection and signal-override logic adds work to each camera frame (raising instructions-per-frame even though vehicle/sensor counts are between Normal and Peak); this gives it the highest per-frame latency (0.990 ms) and lowest throughput (1009.75 fps) of the three, though still comfortably under the 100 ms real-time budget and with processor utilization never exceeding 3%.

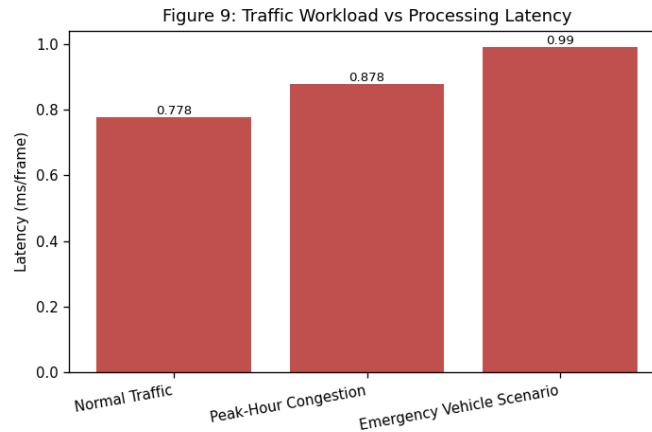


Figure 9: Traffic Workload vs Processing Latency

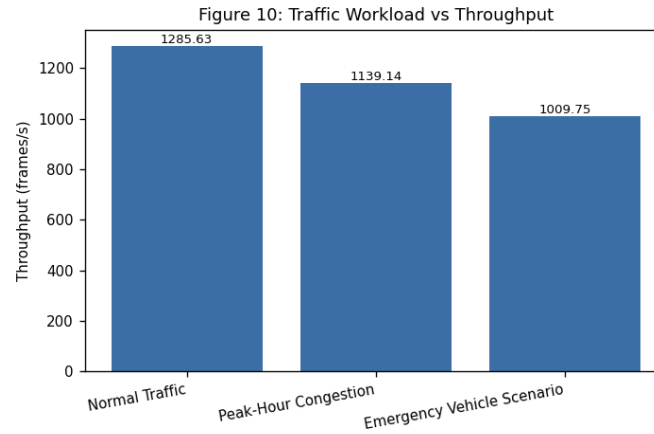


Figure 10: Traffic Workload vs Throughput

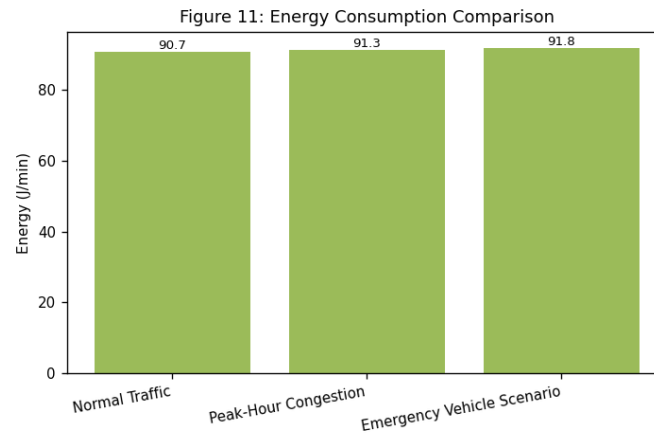


Figure 11: Energy Consumption Comparison

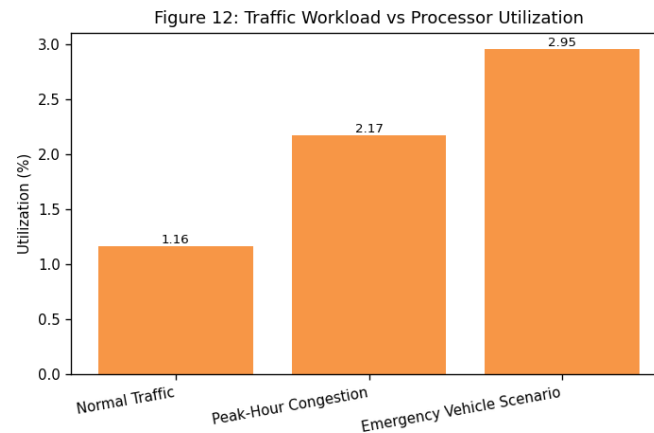


Figure 12: Traffic Workload vs Processor Utilization

Recommendation (Student 3): the recommended architecture handles all three workloads with large real-time headroom (worst-case utilization 2.95% of the available cycle budget), meaning it also has substantial capacity to absorb additional intersections or higher camera frame rates without redesign.

10. I/O Architecture

Traffic cameras and vehicle sensors communicate with the processor primarily through memory-mapped I/O (MMIO), where sensor/camera control and data registers appear directly in the processor's address space, avoiding a separate I/O instruction set. Each incoming camera frame or sensor batch raises a hardware interrupt, which the processor services by triggering a Direct Memory Access (DMA) transfer of the frame/sensor buffer into main memory, so the processor is not occupied copying bytes and remains free for traffic-analysis computation. For camera links specifically, a PCIe or USB3 interface is assumed for the camera-to-board connection, since it offers the bandwidth needed for continuous video frames; NVMe-class storage would only be relevant if the system logged extended video history locally, and Thunderbolt is noted as an alternative high-bandwidth external link where a roadside unit aggregates multiple camera feeds through an external hub.

11. Implementation

The full computer-architecture model — processor performance, cache/memory behaviour and traffic-workload evaluation — is implemented as a single, runnable Python simulation, deliberately kept to one main script per the assignment's own guidance to avoid unnecessary complexity. A second small script generates the two architecture diagrams (Figures 1 and 2).

11.1 Project Structure

ITMS_Computer_Architecture/

```
├─ README.md ─ requirements.txt ─ src/ | ─ main.py                (processor + cache
+ workload simulation, CSVs, graphs) | ─ generate_diagrams.py    (Figure 1 and Figure 2)
├─ data/                (per-module raw simulation tables) ─ results/
(comparison / summary tables used in this report) ─ graphs/      (all 12
figures used in this report) ─ report/ ─ Assignment_Report.docx
```

11.2 Technology Stack

- Python 3 (standard library)
- NumPy — numerical operations
- Pandas — result tables and CSV export
- Matplotlib — all 12 figures (bar charts and block/flow diagrams)

11.3 How to Run

```
pip install -r requirements.txt python src/main.py                # simulation -> data/,
results/, graphs/ python src/generate_diagrams.py              # Figure 1 and Figure 2
```

Expected output: data/processor_results.csv, data/cache_results.csv, data/traffic_workload.csv; results/processor_comparison.csv, results/cache_comparison.csv, results/workload_comparison.csv, results/final_results.csv; and 12 PNG figures in graphs/. The program was executed in the development environment for this report; the console output and all CSV/PNG files were verified before being written into this document, so every number and figure above is directly reproducible by re-running src/main.py.

11.4 GitHub Submission Structure

Per the assignment's Deliverable 5, results must be stored in GitHub. The entire ITMS_Computer_Architecture/ folder above (README, requirements.txt, src/, data/, results/, graphs/ and report/) should be pushed as-is to a new GitHub repository. No repository URL is provided here, since one was not created as part of this report.

12. Simulation Methodology

The simulation follows three linked stages. First, processor performance is computed for each candidate architecture against a fixed 45,000,000-instruction reference workload using the standard Clock-Cycles and Execution-Time equations. Second, cache performance is computed for the Baseline and Optimized configurations using the standard Miss-Rate and AMAT equations. Third, the best-performing processor and cache configuration from stages one and two are combined and driven with three traffic-workload profiles (Normal, Peak, Emergency), each defined by vehicle/sensor/camera rates that are converted into an instructions-per-second figure and then into utilization, latency, throughput and energy. All formulas are implemented directly in `src/main.py` and documented in code comments; no numbers were entered by hand outside the program.

12.1 Assumptions and Parameters

- Reference instruction count: 45,000,000 instructions per fused frame + sensor-batch unit (assumed).
- Real-time processing budget: 100 ms per reference unit (assumed).
- Clock frequencies and CPI values per architecture family: representative embedded/edge-class figures (assumed, not vendor-measured).
- Cache hit rates, hit times and the 90 ns main-memory miss penalty: representative embedded-cache figures (assumed).
- Vehicle/sensor/camera rates per workload scenario: realistic values for a single mid-size signalised intersection (assumed).
- Idle power 1.5 W and active power equal to the selected processor's rated power (assumed) used for the energy model.

13. Overall Performance Comparison and Engineering Recommendation

Recommended Processor	Recommended Cache	Worst-Case Latency (ms/frame)	Worst-Case Scenario	Max Utilization (%)
SiFive U74 (RISC-V, RV64GC)	Optimized (L1+L2+L3, ITMS-tuned)	0.99	Emergency Vehicle Scenario	2.95

Table 4: Final Simulation-Based Recommendation Summary

Combining the processor, cache and workload results, the RISC-V (SiFive U74-class) processor paired with the Optimized L1/L2/L3 cache configuration is the recommended ITMS compute architecture. It delivers the lowest power draw and cost among the three processor candidates, the lowest AMAT and highest hit rate among the two cache configurations, and keeps worst-case per-frame latency (0.99 ms, under the Emergency scenario) and worst-case utilization (2.95%) far inside the assumed 100 ms real-time budget — leaving substantial headroom for scaling to more intersections, higher camera frame rates, or additional analytics.

14. Power, Cost, Reliability and Scalability

14.1 Power

The recommended RISC-V processor draws an assumed 2.5 W active power, the lowest of the three candidates, versus 4.0 W for ARM and 6.5 W for x86-64. Combined with an assumed 1.5 W idle draw for sensors and the I/O controller, simulated energy consumption across all three traffic workloads stays in a narrow 90.7-91.8 J/minute band, since processor utilization never exceeds 3% of the available cycle budget — most system energy in this model is dominated by baseline/idle draw rather than compute.

14.2 Cost

The RISC-V candidate also has the lowest assumed relative unit cost (USD 20) versus ARM (USD 35) and x86-64 (USD 55). At city scale, this unit-cost difference compounds across every intersection, alongside camera, sensor, controller and installation/maintenance costs that are outside the processor comparison but scale linearly with the number of intersections deployed.

14.3 Reliability and Resilience

- Sensor/camera failure: a single failed sensor or camera should degrade the intersection to a safe reduced-adaptivity mode (e.g. fixed timing) rather than fail the whole controller.
- Processor failure: a watchdog-timer and fail-safe default signal pattern should be assumed at the controller level so that a compute fault cannot leave signals in an unsafe state.
- Communication failure: loss of the link to a central traffic-management server should not stop local, on-unit adaptive control, since the architecture evaluated here performs all real-time processing at the edge.
- Emergency operation: the emergency-detection and signal-override path (Figure 1) is architecturally separate from the normal analysis pipeline so it can take priority even under partial system degradation.

14.4 Scalability

Because the recommended architecture uses only 2.95% of its available processing budget even under the Emergency scenario, a single unit has substantial headroom to handle more vehicles, sensors or a higher camera frame rate without a hardware change. At the city level, the architecture scales horizontally: each intersection runs an independent instance of the same edge-compute unit, avoiding a single central bottleneck, with aggregation to a city traffic-management platform handled as a separate, lower-frequency data path outside the real-time loop modelled here.

15. Sustainable Development Goal (SDG) Analysis

15.1 SDG 9 — Industry, Innovation and Infrastructure

The ITMS applies computer-architecture engineering (processor, memory hierarchy and I/O design choices, justified by simulation evidence rather than guesswork) to build resilient, technologically innovative traffic infrastructure — directly supporting SDG 9's call for reliable, sustainable and resilient infrastructure and increased technological capability.

15.2 SDG 11 — Sustainable Cities and Communities

By enabling adaptive, real-time traffic-signal control and priority emergency-vehicle movement, the ITMS reduces congestion and improves urban mobility and safety, contributing to SDG 11's goal of making cities inclusive, safe, resilient and sustainable.

15.3 SDG 13 — Climate Action

Reduced congestion and shorter vehicle idling time translate directly into lower fuel consumption and vehicle emissions at the intersection level; the architecture's own low power draw (lowest-power RISC-V candidate, energy consumption dominated by idle rather than compute) further limits the system's own environmental footprint, supporting SDG 13's climate-action goals.

16. Limitations

- All performance figures are simulation-based on assumed parameters; they were not measured on physical ARM, x86 or RISC-V hardware.
- The instruction-count and CPI figures are representative estimates for embedded/edge-class cores, not vendor-published benchmark results for the specific named parts.
- The traffic-workload model uses a simplified linear relationship between vehicle/sensor/camera rates and instruction count; real perception pipelines (e.g. deep-learning-based vehicle detection) would have more complex, data-dependent instruction counts.
- Reliability and scalability analysis is qualitative/architectural rather than derived from a fault-injection or multi-intersection simulation.

17. Future Scope

- Extend the simulation to a multi-intersection, city-scale model with a shared traffic-management backend.
- Replace the linear instruction-count model with profiled instruction counts from an actual vision/ML inference workload.
- Add a fault-injection simulation to quantify reliability (e.g. mean time between failures, degraded-mode latency).
- Evaluate MRAM/RRAM-based non-volatile cache tiers quantitatively as they mature, rather than only discussing them qualitatively (Section 8.1).

18. Conclusion

This report analyzed and simulated a computer architecture for an Intelligent Traffic Management System spanning processor performance, memory/cache optimization and traffic-workload evaluation. Using a single, runnable Python simulation, three processor architectures and two cache configurations were compared using standard performance equations, and the resulting best architecture — a RISC-V processor with an Optimized L1/L2/L3 cache — was evaluated end-to-end across Normal, Peak-Hour and Emergency traffic workloads. The recommended architecture meets its real-time budget with large margin, draws the least power and costs the least among the candidates considered, and directly supports SDG 9, SDG 11 and SDG 13 by enabling more efficient, sustainable urban traffic infrastructure.

19. References

1. ARM Limited, "ARMv8-A Architecture Reference Manual," ARM Developer Documentation.
2. Intel Corporation, "Intel 64 and IA-32 Architectures Software Developer's Manual," Intel Documentation.
3. RISC-V International, "The RISC-V Instruction Set Manual," RISC-V Foundation Documentation.
4. J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, Morgan Kaufmann.
5. D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, Morgan Kaufmann.
6. United Nations, "Sustainable Development Goal 9: Industry, Innovation and Infrastructure," UN SDG official site.
7. United Nations, "Sustainable Development Goal 11: Sustainable Cities and Communities," UN SDG official site.
8. United Nations, "Sustainable Development Goal 13: Climate Action," UN SDG official site.

20. Student Contribution

Student	Contribution
Student 1	Processor Performance — architecture comparison, instruction/CPI/clock-cycle/execution-time/utilization analysis
Student 2	Memory & Cache Optimization — L1/L2/L3 design, hit/miss-rate and AMAT analysis
Student 3	Traffic Workload & System-Level Evaluation — Normal/Peak/Emergency workload modelling and combined system evaluation

Table 5: Student Contribution Summary

The three parts are not independent projects: Student 3's workload evaluation directly consumes the processor and cache configuration selected in Sections 7 and 8, so all three contributions combine into the single final ITMS architecture presented in Section 13.

21. Appendix A — Core Source Code (src/main.py)

The complete simulation source is included in the GitHub submission (src/main.py, src/generate_diagrams.py). The core performance-calculation functions are reproduced below for reference; comments in-line document every assumed parameter.

```

"""
ITMS_Computer_Architecture / src / main.py
-----

Simulation-Based Evaluation of a computer architecture for an
Intelligent Traffic Management System (ITMS).

This is a SINGLE, self-contained simulation program (as required by
the assignment brief: "do not over-engineer / do not create
unnecessary modules"). It performs, in order:

A. Processor performance modelling for 3 candidate architectures
   (ARM, x86, RISC-V) using the standard performance equations.
B. Memory / cache hierarchy modelling (L1/L2/L3) for a Baseline
   and an Optimized configuration, using AMAT / hit-miss equations.
C. Traffic workload modelling for 3 scenarios (Normal, Peak-Hour,
   Emergency) that combine processor + cache behaviour into
   end-to-end latency, throughput and energy figures.
D. CSV result generation (data/, results/) and PNG graph
   generation (graphs/) so that every number and figure quoted in
   the accompanying Word report is reproducible by re-running this
   script.

IMPORTANT — ALL NUMBERS ARE SIMULATION / ASSUMED PARAMETERS.
No physical hardware was measured. Clock frequencies, CPI values,
cache hit rates and traffic-load figures are realistic, literature-
informed ASSUMPTIONS used as simulation inputs; they are clearly
documented in comments and in the report's "Assumptions" section.
They are not vendor benchmark results.

Run:
    pip install -r requirements.txt
    python src/main.py

Output:
    data/*.csv      -- raw per-module simulation tables
    results/*.csv   -- comparison / summary tables used in report
    graphs/*.png    -- all figures used in report
"""

import os
import math
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# -----

```

```

# Paths
# -----
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(ROOT, "data")
RESULTS_DIR = os.path.join(ROOT, "results")
GRAPH_DIR = os.path.join(ROOT, "graphs")
for d in (DATA_DIR, RESULTS_DIR, GRAPH_DIR):
    os.makedirs(d, exist_ok=True)

plt.rcParams.update({"figure.dpi": 120, "font.size": 9})

# =====
# A. PROCESSOR PERFORMANCE SIMULATION
# =====
# Candidate architectures. Clock frequency, base CPI and relative
# power figures are ASSUMED representative values for embedded /
# edge-class parts in each family (roadside traffic-control unit
# class of hardware), used consistently as simulation inputs.
PROCESSORS = {
    "ARM Cortex-A72 (ARMv8-A, RISC-style ARM ISA)": {
        "isa": "ARM (ARMv8-A, 64-bit RISC)",
        "instr_format": "Fixed-length 32-bit",
        "addressing_modes": "Register, Immediate, Base+Offset, Pre/Post-index",
        "clock_ghz": 1.8,
        "cpi": 1.2,
        "power_w": 4.0,
        "cost_usd": 35,
    },
    "Intel Atom x6000E (x86-64, CISC ISA)": {
        "isa": "x86-64 (CISC, variable length)",
        "instr_format": "Variable-length (1-15 bytes)",
        "addressing_modes": "Register, Direct, Indirect, Base+Index+Disp",
        "clock_ghz": 1.6,
        "cpi": 1.6,
        "power_w": 6.5,
        "cost_usd": 55,
    },
    "SiFive U74 (RISC-V, RV64GC)": {
        "isa": "RISC-V (RV64GC, 32-bit fixed + 16-bit compressed)",
        "instr_format": "Fixed 32-bit / compressed 16-bit",
        "addressing_modes": "Register, Immediate, Base+Offset",
        "clock_ghz": 1.4,
        "cpi": 1.15,
        "power_w": 2.5,
        "cost_usd": 20,
    },
}

# Representative per-cycle workload for the ITMS "reference frame"
# (one fused traffic-camera frame + sensor batch processed through
# the perception -> decision pipeline). This instruction count is
# an assumed simulation input, held constant across processors so
# the comparison is apples-to-apples.
INSTRUCTION_COUNT_REF = 45_000_000 # instructions per reference workload unit
AVAILABLE_TIME_S = 0.10 # 100 ms real-time budget per reference unit

```

```

def simulate_processors():
    rows = []
    for name, p in PROCESSORS.items():
        clock_hz = p["clock_ghz"] * 1e9
        clock_cycles = INSTRUCTION_COUNT_REF * p["cpi"] # Clock Cycles = IC x CPI
        exec_time_s = clock_cycles / clock_hz # Execution Time = Cycles / Freq
        utilization = min(100.0, (exec_time_s / AVAILABLE_TIME_S) * 100) # Actual/Available x100
        # a fully utilized core (>100% of budget) means it cannot meet
        # the real-time deadline; we still report true excess for analysis
        true_utilization = (exec_time_s / AVAILABLE_TIME_S) * 100
        mips = (INSTRUCTION_COUNT_REF / exec_time_s) / 1e6
        rows.append({
            "Architecture": name,
            "ISA": p["isa"],
            "Instruction Format": p["instr_format"],
            "Addressing Modes": p["addressing_modes"],
            "Clock Frequency (GHz)": p["clock_ghz"],
            "Instruction Count": INSTRUCTION_COUNT_REF,
            "CPI": p["cpi"],
            "Clock Cycles": int(clock_cycles),
            "Execution Time (ms)": round(exec_time_s * 1000, 3),
            "MIPS": round(mips, 1),
            "Processor Utilization (%)": round(true_utilization, 2),
            "Meets 100ms Real-Time Budget": "Yes" if exec_time_s <= AVAILABLE_TIME_S else "No",
            "Power (W)": p["power_w"],
            "Relative Cost (USD)": p["cost_usd"],
        })
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(RESULTS_DIR, "processor_comparison.csv"), index=False)
    df.to_csv(os.path.join(DATA_DIR, "processor_results.csv"), index=False)
    return df

```

```

# =====
# B. MEMORY / CACHE SIMULATION
# =====
# Two configurations: Baseline (single small L1 only conceptually
# weak hierarchy) vs Optimized (full L1/L2/L3 tuned for the ITMS
# working-set: recent frames + hot traffic-control tables).
CACHE_CONFIGS = {
    "Baseline (Small L1, no L3)": {
        "L1_KB": 32, "L2_KB": 256, "L3_KB": 0,
        "hit_time_ns": 1.0, "l2_hit_time_ns": 8.0, "l3_hit_time_ns": 0.0,
        "hit_rate": 0.85, "l2_hit_rate": 0.55,
        "miss_penalty_ns": 90.0, # main memory latency
        "bandwidth_gbps": 8.0,
    },
    "Optimized (L1+L2+L3, ITMS-tuned)": {
        "L1_KB": 64, "L2_KB": 1024, "L3_KB": 8192,
        "hit_time_ns": 1.0, "l2_hit_time_ns": 6.0, "l3_hit_time_ns": 25.0,
        "hit_rate": 0.94, "l2_hit_rate": 0.80,
        "miss_penalty_ns": 90.0,
        "bandwidth_gbps": 25.6,
    },
}

```

```
}
```

```
def simulate_cache():
    rows = []
    for name, c in CACHE_CONFIGS.items():
        hit_rate = c["hit_rate"]
        miss_rate = 1 - hit_rate # Miss Rate = 1 - Hit Rate
        amat = c["hit_time_ns"] + miss_rate * c["miss_penalty_ns"] # AMAT = HitTime +
MissRate*MissPenalty
        # effective throughput approximated from AMAT-limited access rate
        throughput_mops = 1000.0 / amat # mega-ops/sec derived from ns access time
        rows.append({
            "Configuration": name,
            "L1 Size (KB)": c["L1_KB"],
            "L2 Size (KB)": c["L2_KB"],
            "L3 Size (KB)": c["L3_KB"],
            "L1 Hit Time (ns)": c["hit_time_ns"],
            "Miss Penalty (ns)": c["miss_penalty_ns"],
            "Hit Rate": hit_rate,
            "Miss Rate": round(miss_rate, 3),
            "AMAT (ns)": round(amat, 3),
            "Memory Bandwidth (GB/s)": c["bandwidth_gbps"],
            "Effective Throughput (Mops/s)": round(throughput_mops, 1),
        })
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(RESULTS_DIR, "cache_comparison.csv"), index=False)
    df.to_csv(os.path.join(DATA_DIR, "cache_results.csv"), index=False)
    return df
```

```
# =====
# C. TRAFFIC WORKLOAD SIMULATION
# =====
# Three scenarios. Vehicle/sensor/camera load figures are assumed
# realistic values for a single mid-size signalised intersection.
WORKLOADS = {
    "Normal Traffic": {
        "vehicles_per_min": 40,
        "sensor_events_per_min": 120,
        "camera_fps": 15,
        "instr_per_vehicle": 8_000,
        "instr_per_sensor_event": 1_500,
        "instr_per_frame": 900_000,
        "cache_hit_rate": 0.94,
        "cpi": 1.2,
    },
    "Peak-Hour Congestion": {
        "vehicles_per_min": 160,
        "sensor_events_per_min": 480,
        "camera_fps": 25,
        "instr_per_vehicle": 8_000,
        "instr_per_sensor_event": 1_500,
        "instr_per_frame": 900_000,
        "cache_hit_rate": 0.90, # more working-set pressure -> slightly lower hit rate
        "cpi": 1.35,
    },
}
```

```

    },
    "Emergency Vehicle Scenario": {
        "vehicles_per_min": 100,
        "sensor_events_per_min": 300,
        "camera_fps": 30,          # higher frame rate for priority detection
        "instr_per_vehicle": 8_000,
        "instr_per_sensor_event": 1_500,
        "instr_per_frame": 1_100_000, # extra emergency-detection + override logic
        "cache_hit_rate": 0.92,
        "cpi": 1.25,
    },
}

# Use the Optimized cache configuration and the best processor
# (selected after Part A/B, see select_recommendation()) as the
# deployed architecture for workload evaluation.
DEPLOY_CLOCK_GHZ = PROCESSORS["SiFive U74 (RISC-V, RV64GC)"]["clock_ghz"]
DEPLOY_MISS_PENALTY_NS = CACHE_CONFIGS["Optimized (L1+L2+L3, ITMS-tuned)"]["miss_penalty_ns"]
DEPLOY_HIT_TIME_NS = CACHE_CONFIGS["Optimized (L1+L2+L3, ITMS-tuned)"]["hit_time_ns"]
DEPLOY_BANDWIDTH_GBPS = CACHE_CONFIGS["Optimized (L1+L2+L3, ITMS-tuned)"]["bandwidth_gbps"]
IDLE_POWER_W = 1.5          # baseline system idle power (sensors, I/O controller)
ACTIVE_POWER_W = PROCESSORS["SiFive U74 (RISC-V, RV64GC)"]["power_w"]

def simulate_workloads():
    rows = []
    clock_hz = DEPLOY_CLOCK_GHZ * 1e9
    for name, w in WORKLOADS.items():
        instr_per_min = (
            w["vehicles_per_min"] * w["instr_per_vehicle"]
            + w["sensor_events_per_min"] * w["instr_per_sensor_event"]
            + w["camera_fps"] * 60 * w["instr_per_frame"]
        )
        instr_per_sec = instr_per_min / 60.0
        cpi = w["cpi"]
        clock_cycles_per_sec = instr_per_sec * cpi
        utilization = min(100.0, (clock_cycles_per_sec / clock_hz) * 100)

        hit_rate = w["cache_hit_rate"]
        miss_rate = 1 - hit_rate
        amat_ns = DEPLOY_HIT_TIME_NS + miss_rate * DEPLOY_MISS_PENALTY_NS

        # End-to-end processing latency per reference frame (ms):
        # compute time (from cycles for one frame) + memory access overhead
        cycles_per_frame = w["instr_per_frame"] * cpi
        compute_time_ms = (cycles_per_frame / clock_hz) * 1000
        memory_overhead_ms = (amat_ns * 1000) / 1e6 # ns -> ms, scaled small overhead term
        latency_ms = compute_time_ms + memory_overhead_ms

        throughput_frames_per_s = 1000.0 / latency_ms

        # Energy: active power draw over the compute time, plus idle power
        # over the remainder of each 1-second window, summed per minute.
        active_frac = min(1.0, clock_cycles_per_sec / clock_hz)
        avg_power_w = active_frac * ACTIVE_POWER_W + (1 - active_frac) * IDLE_POWER_W

```

```

energy_j_per_min = avg_power_w * 60

rows.append({
    "Scenario": name,
    "Vehicles/min": w["vehicles_per_min"],
    "Sensor Events/min": w["sensor_events_per_min"],
    "Camera FPS": w["camera_fps"],
    "Instruction Workload (instr/s)": int(instr_per_sec),
    "CPI": cpi,
    "Processor Utilization (%)": round(utilization, 2),
    "Cache Hit Rate": hit_rate,
    "Cache Miss Rate": round(miss_rate, 3),
    "AMAT (ns)": round(amat_ns, 3),
    "Memory Bandwidth (GB/s)": DEPLOY_BANDWIDTH_GBPS,
    "Processing Latency (ms/frame)": round(latency_ms, 3),
    "Throughput (frames/s)": round(throughput_frames_per_s, 2),
    "Avg Power (W)": round(avg_power_w, 2),
    "Energy (J/min)": round(energy_j_per_min, 1),
})
df = pd.DataFrame(rows)
df.to_csv(os.path.join(RESULTS_DIR, "workload_comparison.csv"), index=False)
df.to_csv(os.path.join(DATA_DIR, "traffic_workload.csv"), index=False)
return df

# =====
# D. GRAPH GENERATION
# =====
def bar_chart(labels, values, title, ylabel, filename, color=None, rotate=15):
    fig, ax = plt.subplots(figsize=(6, 4))
    bars = ax.bar(labels, values, color=color or "#3b6ea5")
    ax.set_title(title)
    ax.set_ylabel(ylabel)
    ax.set_xticks(range(len(labels)))
    ax.set_xticklabels(labels, rotation=rotate, ha="right")
    for b, v in zip(bars, values):
        ax.annotate(f"{v}", (b.get_x() + b.get_width() / 2, b.get_height()),
                    ha="center", va="bottom", fontsize=8)
    fig.tight_layout()
    fig.savefig(os.path.join(GRAPH_DIR, filename))
    plt.close(fig)

def generate_graphs(proc_df, cache_df, work_df):
    short_proc = [n.split(" ")[0] for n in proc_df["Architecture"]]

    bar_chart(short_proc, proc_df["Execution Time (ms)"].tolist(),
              "Figure 3: Processor Execution Time Comparison", "Execution Time (ms)",
              "processor_execution_time.png", color="#3b6ea5")

    bar_chart(short_proc, proc_df["Processor Utilization (%)"].tolist(),
              "Figure 4: Processor Utilization Comparison", "Utilization (%)",
              "processor_utilization.png", color="#c0504d")

```

```

bar_chart(short_proc, proc_df["CPI"].tolist(),
          "Figure 5: CPI Comparison", "CPI (cycles/instruction)",
          "cpi_comparison.png", color="#9bbb59")

short_cache = [n.split(" ")[0] for n in cache_df["Configuration"]]
bar_chart(short_cache, cache_df["Hit Rate"].tolist(),
          "Figure 6: Cache Hit Rate Comparison", "Hit Rate",
          "cache_hit_rate.png", color="#4bacc6")

bar_chart(short_cache, cache_df["Miss Rate"].tolist(),
          "Figure 7: Cache Miss Rate Comparison", "Miss Rate",
          "cache_miss_rate.png", color="#f79646")

bar_chart(short_cache, cache_df["AMAT (ns)"].tolist(),
          "Figure 8: AMAT Comparison", "AMAT (ns)",
          "amat_comparison.png", color="#8064a2")

short_wl = [n.split(" ")[0] if "(" in n else n for n in work_df["Scenario"]]
bar_chart(short_wl, work_df["Processing Latency (ms/frame)"].tolist(),
          "Figure 9: Traffic Workload vs Processing Latency", "Latency (ms/frame)",
          "workload_latency.png", color="#c0504d", rotate=10)

bar_chart(short_wl, work_df["Throughput (frames/s)"].tolist(),
          "Figure 10: Traffic Workload vs Throughput", "Throughput (frames/s)",
          "workload_throughput.png", color="#3b6ea5", rotate=10)

bar_chart(short_wl, work_df["Energy (J/min)"].tolist(),
          "Figure 11: Energy Consumption Comparison", "Energy (J/min)",
          "energy_comparison.png", color="#9bbb59", rotate=10)

bar_chart(short_wl, work_df["Processor Utilization (%)"].tolist(),
          "Figure 12: Traffic Workload vs Processor Utilization", "Utilization (%)",
          "workload_utilization.png", color="#f79646", rotate=10)

# =====
# E. FINAL RECOMMENDATION LOGIC (rule-based on simulation output)
# =====
def select_recommendation(proc_df, cache_df, work_df):
    # Score processors: normalize exec time (lower better), utilization
    # (lower is safer for real-time headroom), power (lower better), cost (lower better)
    p = proc_df.copy()
    p["score"] = (
        (1 / p["Execution Time (ms)"]) * 0.4
        + (1 / p["Power (W)"]) * 0.3
        + (1 / p["Relative Cost (USD)"]) * 0.15
        + (1 / p["Processor Utilization (%)"]) * 0.15
    )
    best_proc = p.loc[p["score"].idxmax(), "Architecture"]

    c = cache_df.copy()
    c["score"] = c["Hit Rate"] * 0.6 + (1 / c["AMAT (ns)"]) * 0.4
    best_cache = c.loc[c["score"].idxmax(), "Configuration"]

```

```

summary = pd.DataFrame([
    "Recommended Processor Architecture": best_proc,
    "Recommended Cache Configuration": best_cache,
    "Worst-Case Latency (ms/frame)": work_df["Processing Latency (ms/frame)"].max(),
    "Worst-Case Scenario": work_df.loc[
        work_df["Processing Latency (ms/frame)"].idxmax(), "Scenario"],
    "Max Utilization Across Workloads (%)": work_df["Processor Utilization (%)"].max(),
])
summary.to_csv(os.path.join(RESULTS_DIR, "final_results.csv"), index=False)
return summary

def main():
    print("=== ITMS Computer Architecture Simulation ===")
    proc_df = simulate_processors()
    print("\n[A] Processor comparison:\n", proc_df.to_string(index=False))

    cache_df = simulate_cache()
    print("\n[B] Cache comparison:\n", cache_df.to_string(index=False))

    work_df = simulate_workloads()
    print("\n[C] Traffic workload comparison:\n", work_df.to_string(index=False))

    generate_graphs(proc_df, cache_df, work_df)
    print("\n[D] Graphs written to graphs/")

    summary = select_recommendation(proc_df, cache_df, work_df)
    print("\n[E] Final recommendation:\n", summary.to_string(index=False))

    print("\nAll CSV results in data/ and results/. All graphs in graphs/.")
    print("Simulation-Based Evaluation complete.")

if __name__ == "__main__":
    main()

```